CO1_SentiAnalysisP3

May 30, 2025

Dela Cruz, Cloie Sofia - Morales, Rayleen Claire / AM1

IMPORT LIBRARIES

[1]:
```
# This line of code will import the pandas library into the jupyter notebook.
# Pandas is a python package that we will use to analyse and manipulate our␣
 ↪dataset.
# Using the 'as' syntax, we were able to assign the alias 'pd' to panda to␣
 ↪reference its functions easier throughout the script.

import pandas as pd
```

[2]:
```
#This line of code will import the numpy library.
#NumPy is a library in Python for the computation and manipulation of data.

import numpy as np
```

[3]:
```
#Importing train_test_split is useful for data splitting later.
#This makes dividing the training and testing data more convenient.

from sklearn.model_selection import train_test_split
```

[4]:
```
#Importing TfidVectorizer is useful for vectorizing the train and test data.
#Tokenization will also be applied while using this import.

from sklearn.feature_extraction.text import TfidfVectorizer
```

[5]:
```
#From here, it is evident what algorithm we have decided to use for training␣
 ↪our model which is Logistic Regression.
#We chose this model as it is simple and reliable for classifying given factors.
#This will later be useful in training our model for future predictions of test␣
 ↪data.

from sklearn.linear_model import LogisticRegression
```

[6]:
```
#The accuracy_score import is for showing how our model has accurately␣
 ↪predicted the sentiments of the tweets.
```

```
#The classification_report import is for detailing the performance of our model␣
 ↪through precision, recall, and its f1-score.

from sklearn.metrics import accuracy_score, classification_report
```

[7]:
```
#NLTK or Natural Language Toolkit is an import helpful for Natural Language␣
 ↪Processing (NLP), which works with human language data (NLTK, 2009).
#This library will be used for preprocessing our data for later.

import nltk
```

[8]:
```
# This line downloads the 'punkt' tokenizer model from nltk library.
# This model can help split the tweets into seprate words.

nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     /home/rayleen_morales/nltk_data…
[nltk_data]   Package punkt is already up-to-date!
```

[8]:
```
True
```

[9]:
```
# This line of code downloads the 'stopwords' dataset from nltk library.
# These stopwords are common words in a language that are irrelevant for our␣
 ↪sentiment analysis.

nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     /home/rayleen_morales/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
```

[9]:
```
True
```

[10]:
```
#This import holds the list of stopwords to help filter out data for later.
#Removal of stopwords is part of the data preprocessing to easily analyze␣
 ↪significant words and ignore common, non-comprehensive data.

from nltk.corpus import stopwords
```

[11]:
```
#This import is part of the stopwords removal which splits the string into␣
 ↪words.
#This comes hand-in-hand with the filtering of stopwords and rejoining the␣
 ↪string to store into a new column as the new preprocessed data.

from nltk.tokenize import word_tokenize
```

[12]:
```
#This module holds predefined strings such as ASCII letters, digits, and even␣
 ↪whitespaces.
#For this activity, this import will be used to filter the punctuations and␣
 ↪other special characters in the data later.

import string
```

[13]:
```
# This line of code loads the Seaborn library into the notebook.
# The Seaborn library is useful for statistical plotting in python by providing␣
 ↪default styles and color palettes (GeeksForGeeks, 2024d).

import seaborn as sns
```

[14]:
```
# This line of code imports the 'confusion_matrix' function from scikit-learn's␣
 ↪'metrics' module.
# The confusion matrix will help us evaluate how well our model performs by␣
 ↪showing the number of correct and incorrect prediction made by the model by␣
 ↪each class.

from sklearn.metrics import confusion_matrix
```

[15]:
```
# The 'pyplot' module from Matplotlib provides a simple way to create charts␣
 ↪and graphs (W3Schools, 2025c).
# While Seaborn draws the plot, Matplotlib (plt) will show it, and add labels␣
 ↪and formatting.

import matplotlib.pyplot as plt
```

[PART 1] LOAD DATASET

[16]:
```
# This line of code loads our dataset, which is a CSV file named␣
 ↪'twitter_training.csv,' into a DataFrame called 'df.'
# According to W3Schools, a DataFrame is a 2D data structure which can be␣
 ↪visualized with a table with rows and columns.
# The function pd.read_csv() reads our dataset and automatically determines the␣
 ↪column headers and data types for the creation of the DataFrame df.

df = pd.read_csv('twitter_training.csv')
```

INITIAL UNDERSTANDING

[17]:
```
# The function head() displays the first five rows of the df DataFrame.
# This is a usual way for us to quickly inspect the column headers and their␣
 ↪content.
# Through this, we are able to get a quick understanding of the structure of␣
 ↪the data we are working with without loading all of it into memory.
```

```
df.head()
```

[17]:

	label	premise
0	Positive	im getting on borderlands and i will murder yo…
1	Positive	I am coming to the borders and I will kill you…
2	Positive	im getting on borderlands and i will kill you …
3	Positive	im coming on borderlands and i will murder you…
4	Positive	im getting on borderlands 2 and i will murder …

[18]:
```
# The shape() function is used to return a tuple showing the dimensionality of the DataFrame df (pandas, 2024).
# The first value 74 682 is the number of records in our dataset (rows), while the second value 2 is the number of columns.
df.shape
```

[18]: (74682, 2)

[19]:
```
# The info() function from the pandas library prints more information about our df DataFrame (pandas, 2024).
# It includes information like the numbers of rows (indexed from 0 to 74 681), the number of columns (2), the different column names ('label' and 'premise'), the number of non-null rows, the data type of each column ('object'), and the total memory usage of the dataset in RAM.
# It is important to note that the number of non-null values in the premise column is larger than the total number of records. This means that there are some values in the premise column that are null and needs to be cleaned later.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74682 entries, 0 to 74681
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype 
---  ------   --------------  ----- 
 0   label    74682 non-null  object
 1   premise  73996 non-null  object
dtypes: object(2)
memory usage: 1.1+ MB
```

[20]:
```
# This line of code is used to list all column names in the DataFrame df.
# Through this, we were able to confirm that expected columns like 'label' and 'premise' are present in the DataFrame.

df.columns
```

[20]: Index(['label', 'premise'], dtype='object')

[21]:
```
# This prints all the unique values in the 'label' column, identifying all␣
 ↪sentiment categories in the dataset.
# For the purposes of this analysis, we have elected to delete all records with␣
 ↪'irrelevant' values because they do not contribute to our task.

print(df['label'].unique())
```

```
['Positive' 'Neutral' 'Negative' 'Irrelevant']
```

[22]:
```
# This line of code counts (with value_counts() function) and prints (with␣
 ↪print() function) the frequency of each label in the 'label' column.
# From its results, we can derive any imabalance in our sentiment␣
 ↪classification.

print(df['label'].value_counts())
```

```
label
Negative      22542
Positive      20832
Neutral       18318
Irrelevant    12990
Name: count, dtype: int64
```

[23]:
```
# This line of code filters out rows where the label is 'irrelevant.'
# We used the function str.lower() to ignore case sensitivity in the labels.
# This is an important cleaning step so that our dataset is only focused on␣
 ↪valid sentiment categories.

df = df[df['label'].str.lower() != 'irrelevant']
```

[24]:
```
# After filtering, this line of code reprints the count of each remaining␣
 ↪records of the sentiment labels.
# Through this, we can verify that all 'irrelevant' records were successfully␣
 ↪removed by the above code.

print(df['label'].value_counts())
```

```
label
Negative    22542
Positive    20832
Neutral     18318
Name: count, dtype: int64
```

[25]:
```
# The reset_index() function resets the DataFrame's index so that the row␣
 ↪numbers are sequential after filtering.
# The 'drop=True' parameter prevents the old index from being added as a new␣
 ↪column.
```

```
# On the other hand, the 'inplace=True' applied the changes directly to the df␣
 ↪DataFrame.

df.reset_index(drop=True, inplace=True)
```

[26]:
```
# The shape function will show the number of rows and columns in the df␣
 ↪DataFrame.
# The number of rows confirms that the 'irrelevant' records were filtered out␣
 ↪and that the dataset maintains its two columns.

df.shape
```

[26]:
```
(61692, 2)
```

[27]:
```
# After running the info() function again, we noticed the large number of null␣
 ↪values in the premise column.
# Because of this, we have elected to filter out these records from the df␣
 ↪DataFrame.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61692 entries, 0 to 61691
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    61692 non-null  object
 1   premise  61121 non-null  object
dtypes: object(2)
memory usage: 964.1+ KB
```

[28]:
```
# This line of code displays the records with null values in the premise column.
# The 'df['premise']' part specifies the relevant column, while the 'isnull()␣
 ↪function specifies that only null values are to be displayed.

df[df['premise'].isnull()]
```

[28]:

	label	premise
61	Neutral	NaN
493	Neutral	NaN
511	Neutral	NaN
661	Positive	NaN
985	Positive	NaN
…	…	…
60994	Positive	NaN
60995	Positive	NaN
61431	Positive	NaN

```
61432  Positive     NaN
61433  Positive     NaN

[571 rows x 2 columns]
```

[29]:
```
# The function dropna() removes all rows from the df DataFrame where the value in the 'premise' column is 'NaN.'
# The subset parameter ensure that only the 'premise' columns is checked for any null values and the other column 'label' is not checked for null values.
# This line also assigns the results back to the df DataFrame to filter out and records with missing text data.

df = df.dropna(subset=['premise'])
```

[30]:
```
# Similar to its usage above, the reset_index() function resets the indeces in the df DataFrame.
# The parameters 'drop' and 'inplace' also makes sure to avoid adding the old index as a new column in the DataFrame and applies the changes to the DataFrame itself.

df.reset_index(drop=True, inplace=True)
```

[31]:
```
# This final line of code provides a short summary of the different features of our df DataFrame.
# There are now less entries because we cleaned our dataset from null values and 'irrelevant' records.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61121 entries, 0 to 61120
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype 
---  ------   --------------  ----- 
 0   label    61121 non-null  object
 1   premise  61121 non-null  object
dtypes: object(2)
memory usage: 955.1+ KB
```

[PART 2] DATA PREPROCESSING

[32]:
```
#This line of code converts all tweets under the columns 'premise' to lowercase.
#Lowercasing unifies data, reducing the need to process the text's tone from its casing (Vanga, 2024).

df['premise'] = df['premise'].str.lower()
```

[33]:
```
#This function "str.replace()" aims to remove punctuation and special␣
 ↪characters through replacing "f"[{string.punctuation}]"" with a blank string␣
 ↪("").
#The parameter "regex=True" considers the "string.punctuation" as a regular␣
 ↪expression for easier pattern matching/searching within strings (Yang, 2020).
#Removal of punctuation further reduces tone ambiguity from its characters␣
 ↪(Vanga, 2024).

df['premise'] = df['premise'].str.replace(f"[{string.punctuation}]", "",␣
 ↪regex=True)
```

[34]:
```
#The English set of stopwords from the NLTK library is stored in the variable␣
 ↪'stop_words' (Geeksforgeeks, 2024).
#This set will be used later to filter out the stopwords under the column␣
 ↪'premise'.

stop_words = set(stopwords.words('english'))
```

[35]:
```
#This code applies a function to all rows of the 'premise' column to remove␣
 ↪stopwords from each tweet.
#For each row, the tweet is split into individual words, stopwords are filtered␣
 ↪out, and remaining words are rejoined into a string (Geeksforgeeks, 2024a).
#The new cleaned string is stored in a new column 'cleaned'.

df['cleaned'] = df['premise'].apply(lambda x: ' '.join(word for word in␣
 ↪word_tokenize(x) if word.lower() not in stop_words))
```

[36]:
```
#This code simply prints the data frame to observe and compare its preprocessed␣
 ↪and cleaned data before splitting it.
#The 'cleaned' column in here is not present in the CSV file itself, but only␣
 ↪in the dataframe.

print(df)
```

```
          label                                              premise  \
0      Positive  im getting on borderlands and i will murder yo…
1      Positive  i am coming to the borders and i will kill you…
2      Positive  im getting on borderlands and i will kill you all
3      Positive  im coming on borderlands and i will murder you…
4      Positive  im getting on borderlands 2 and i will murder …
…         …                                                   …
61116  Positive  just realized that the windows partition of my…
61117  Positive  just realized that my mac window partition is …
61118  Positive  just realized the windows partition of my mac …
61119  Positive  just realized between the windows partition of…
61120  Positive  just like the windows partition of my mac is l…
```

```
                                                  cleaned
0                              im getting borderlands murder
1                                        coming borders kill
2                                im getting borderlands kill
3                               im coming borderlands murder
4                             im getting borderlands 2 murder
…                                                       …
61116  realized windows partition mac like 6 years be…
61117  realized mac window partition 6 years behind n…
61118  realized windows partition mac 6 years behind …
61119  realized windows partition mac like 6 years be…
61120  like windows partition mac like 6 years behind…

[61121 rows x 3 columns]
```

DATA SPLITTING

[37]:
```
#This code assigns the 'cleaned' column to variable 'X' to prepare for data␣
 ↪splitting later.
#This column contains the cleaned and preprocessed tweets that have gone␣
 ↪through lowercasing and punctuation and stopwords removal.

X = df['cleaned']
```

[38]:
```
#This code assigns the 'label' column to variable 'y' to prepare for data␣
 ↪splitting later.
#This column contains the sentiments for each tweet, in which the model will␣
 ↪learn and aim to predict later.

y = df['label']
```

[39]:
```
#The function train_test_split() splits the data (X and y) into train and test␣
 ↪subsets, with 80% for train (train_size=0.8) and 20% for test (test_size=0.
 ↪2).
#The random_state parameter is for data shuffling before the split and stratify␣
 ↪helps split the data in proportion to the 'label' column (Scikit Learn,␣
 ↪2018).
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪train_size=0.8, random_state=3, stratify=y)
```

FEATURE EXTRACTION

[40]:
```
#This initializes the vectorizer by creating an instance.
#The instance will be used to fit and transform the cleaned data later.
vectorizer = TfidfVectorizer()
```

[41]:
```
#The 'fit_transform' method helps learn the vocabulary and the IDF from the␣
 ↪corpus.
```

```
#This then return the TF-IDF matrix to the variable X (Geeksforgeeks, 2024).
#TF-IDF stands for Term Frequency - Inverse Document Frequency which helps in␣
 ↪scoring a word.
#This reveals the importance of a word by contrasting the commonality and␣
 ↪rarity of a word (Karabiber, 2024).

X_train_trans = vectorizer.fit_transform(X_train)
```

[42]:
```
#The 'transform' method helps transform our test data in its standardized form␣
 ↪(Khanna, 2020).
#This applies the vocabulary learned from the training data.

X_test_trans = vectorizer.transform(X_test)
```

TRAINING THE MODEL

[43]:
```
#Logistic Regression is used for our model training, which is commonly known␣
 ↪for its simple and reliable binary classification tasks through relationship␣
 ↪analysis of given variables (Geeksforgeeks, 2024b).
#The maximum number of iterations is set to 10,000 to find the most accurate␣
 ↪sentiment (label) prediction from the provided tweets (premises) (Scikit␣
 ↪Learn, 2025).

model = LogisticRegression(max_iter=10000)
```

[44]:
```
#The fit() method is the training part of the model using the training data.
#The model learns from this data to prepare itself for later predictions.

model.fit(X_train_trans, y_train)
```

[44]:
```
LogisticRegression(max_iter=10000)
```

MAKING PREDICTIONS

[45]:
```
#The predict() methods is the prediction part of the model using the test data.
#Resulting predictions is stored in y_pred for performance evaluation later.

y_pred = model.predict(X_test_trans)
```

[PART 3] MODEL EVALUATION

[46]:
```
#This prints the accuracy score of the model based on its predictions (y_pred)␣
 ↪compared with the original labels (y_test).
#The accuracy score of our model is 0.8259304703476482, which reveals an 82.59%␣
 ↪accuracy for this dataset.

print(accuracy_score(y_test, y_pred))
```

```
0.8259304703476482
```

[47]:
```
#This classification report reveals that our model predicts best on 'negative' tweets with the highest f1-score of 0.85, 'positive' predictions are high as well, with 0.83.
#However, this leaves 0.79 for the neutral tweets, which also reveals it struggles to identify this sentiment and would need further improvements.

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

    Negative       0.84      0.86      0.85      4472
     Neutral       0.80      0.79      0.79      3622
    Positive       0.84      0.82      0.83      4131

    accuracy                           0.83     12225
   macro avg       0.82      0.82      0.82     12225
weighted avg       0.83      0.83      0.83     12225
```

[48]:
```
# This line of code computes a confusion matrix by comparing y_test (the true label) with the y_pred (the labels predicted by our model.
# This results in 'cm' which is a 2D array (matrix) that tell us how many times the model predicted each class and if it was correct.
# According to GeeksForGeeks (2025), a confusion matrix can measure how well a classification model is performing and show where the model is making mistakes.

cm = confusion_matrix(y_test, y_pred)
```

[49]:
```
# Using the 'figsize()' attribute of the 'figure()' function, we can change the size of the plot.
# The first parameter of figsize() takes in the width while the other parameter is the height (Abba, 2023).
plt.figure(figsize=(8,6))

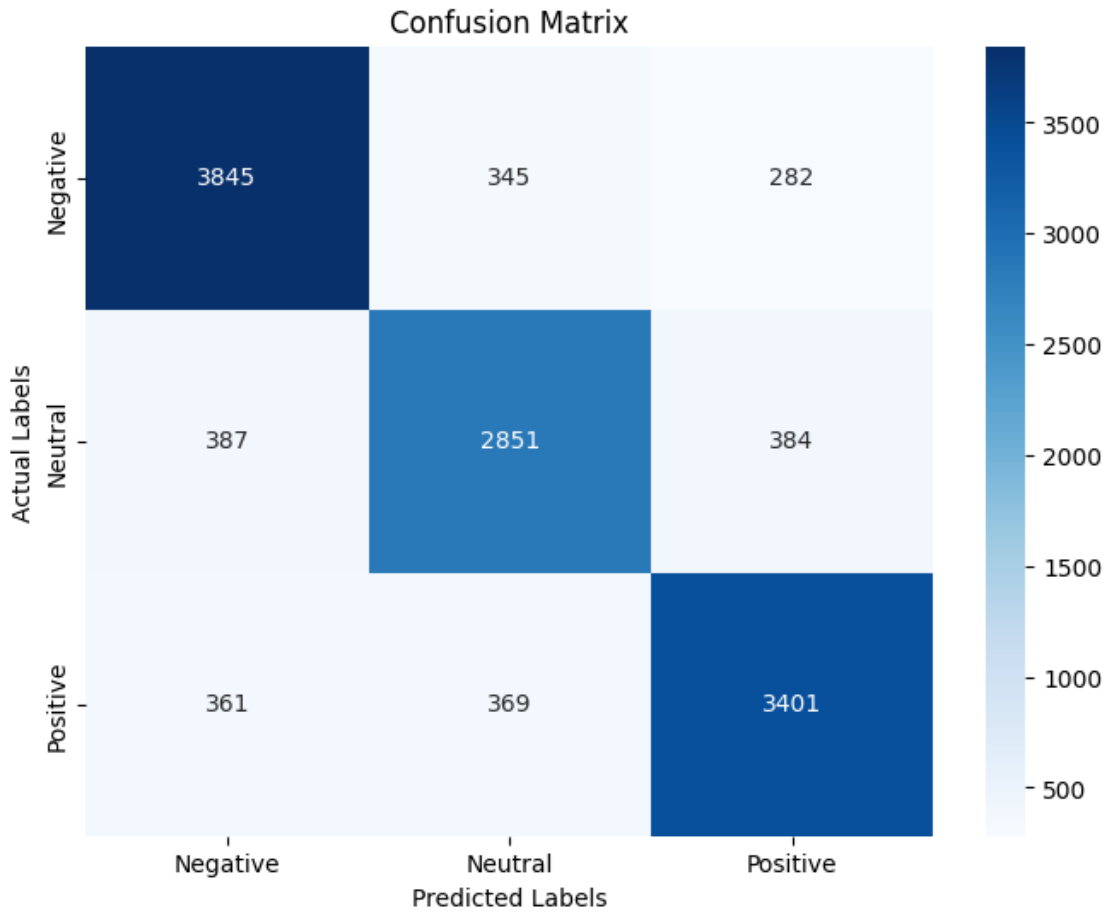
# The 'heatmap()' function plots the 2D dataset cm as a color-encoded matrix. It's first parameter takes in the actual data to be turned into a matrix.
# The 'annot=True' tells the function to write the data value in each cell, while 'fmt='d'' formats the numbers as integers.
# Additionally, 'cmap='Blues'' sets the color gradient to shades of blue with the darker shades indicating a higher number of predictions.
# Lastly, the last two parameters labels the X (columns) and Y (rows) axes using the actual class names (negative, neutral, and positive) (Seaborn, 2024).
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=model.classes_, yticklabels=model.classes_)
```

```
# This line of code adds a 'Predicted Labels' label below the x axis.
# This can be helpful to know the meaning of the columns in the matrix.
plt.xlabel("Predicted Labels")

# This line of code adds a 'Actual Labels' label to the y-axis.
# These are the true labels from the test set.
plt.ylabel("Actual Labels")

# This line of code adds a title on top of the plot.
# The 'Confusion Matrix' title helps in identifying the matrix which is␣
 ↪especially helpful when we are dealing wiht more than one.
plt.title("Confusion Matrix")

# This line of code displays the plot in our notebook.
# Without this, the plot might not appear.
plt.show()
```



INTERPRETING EVALUATION RESULTS

Accuracy Score

An accuracy score of 0.8259304703476482 means that roughly 82.6% of our model's label predictions were correct. Although this may be considered a good score for sentiment analysis (Lexanalytics, 2024), this figure can be misleading if ever our classes are imbalanced, making other evaluation tools like classification reports and confusion matrix important.

Classification Report

Starting with the Negative class, the model exhibits strong precision (84%). This indicates that when it predicts a tweet as Negative, it is correct 84% of the time. Its recall (86%) means its successfully identifies 86% of all actual Negative tweets. Meanwhile, its F1-score (85%), which balances precision and recall, reflects the model's consistent and reliable performance in deetecting the negative sentiment. The classification report also tells us that the Negative class is the most represented class in the test set with 4472 instances. This possibly contributes to the model's strong performance with the Negative class because it was more exposed to this type of sentiment during training.

Moving on, the model predicts Positive tweets with 84% precision. This means its positive prediction are correct most of the time. The recall (82%) indicates that the model also successfully identifies 82% of actual Positive tweets. Lastly, the F1-score of 83% reflects a solid balance between the precision and recall, showing the consistent performance of the model on Positive tweets. Notably, the Positive class has 4131 samples in the test set. Although it is lesser than the number of Negative tweets, this amount is still enough to support and train the model efficiently.

The neutral class has a notably lower precision with only 80%, meaning that 80% of the tweets the model labeled as Neutral were truly neutral. The recall (79%) show that the model only captures 79% of actual Neutral tweets. It misses about 21% of them, possibly misclassifying them as Positive or Negative. Notably, the F1-score (79%) of the neutral class is the lowest among the three classes, indicating that the Neutral is the most challenging class for the model. It is important to remember that neutral tweets often contain less emotionally charged language, making them harder to distinguish from either end of the sentiment spectrum.

Lastly, the overall model performance in accuracy is 83% which confirms that the model correctly predicts the sentiment of roughly 83 tweets out of every 100. This is consistent with the earlier result in the accuracy score. The macro average (0.82) tells us that the model treats each class equally and shows balanced performance across all classes despite differences in class size. The weighted average (0.83) takes into account the number of samples per class and reflects an overall strong performance.

Confusion Matrix

The confusion matrix reveals that the model performs most accurately in classifying Negative tweets. Only a small portion of Negative tweets were misclassified as Neutral (345) or Positive (282), suggesting that the model effectively distinguishes strongly negative sentiment from other categories. These results aligns with the high precision and recall values previously observed in the classification report.

For Positive tweets, the model also demonstrates a strong and consistent performance. Out of 4131 Positive samples, the model correctly classified 3401, resulting in an 82.3% accuracy for this class. The remaining instances were mostly confused with Neutral (369) and Negative (361). This tells us

that while the model is generally accurate, it is still confused when distinguishing between Neutral and mildy Positive language. These results also reinforces the classification's findings.

In contrast, the model exhibits the most difficulty in classifying Neutral tweets. Of the 3622 actual Neutral samples, only 2851 were correctly identified, while 387 were mistakenly labeled as Negative and 384 as Positive tweets. This suggests that the model struggles to distinguish Neutral sentiment. This is likely because neutral language tends to share vocabulary with both positive and negative expressions. Notably, the almost even split of confusion between Negative and Positive highlights that that Neutral sentiments may lack the emotional or polarizes flags that the model relies on fo accurate classification.

Overall, the confusion matrix echoes the results of the accuracy score and the classification report. The strong diagonal tells use that most predictions are correct. However, the neutral class is heavily misclassified with the model confused with both Positive and Negative. On the brighter note, the confusion matrix shows similar number of predictions across classes, telling us that the model isn't heavility biased toward any one class.

STRENGTHS AND WEAKNESSES OF THE MODEL

Based on our accuracy score, classification report, and confusion matrix, our model's strengths are the following:

1. High Overall Accuracy- Our model's accuracy score of 82.59% is strong for a multi-class text classification model. This tells us that the model makes correct predictions for over 8 out of every 10 tweets.
2. High F1-score in Negative and Positive Classes - Both the Negative and the Positive classes demonstrate high precision and f1-scores, telling us that the model reliably identifies these sentiments (as reflected in the confusion matrix).
3. No bias towards one class - The model demonstrates balanced prediction between Positive and Negative classes. This is seen with the weighted average metric remaining consistent with the overall accuracy, meaning that the class imbalance is not skewing the results severely.

On the other hand, our model's weaknesses are the following:

1. Lowest F1-score in the Neutral class - The most prominent weakness of the model is classifying the Neutral tweets. With the lowerst F1-score and over 20% of Neutral tweets misclassified, the model has difficulty in distinguishing Neutral sentiment.
2. Neutral tweets are frequently confused with both Positive and Negative - The confusion matrix tells us that the Neutral tweets are frequently confused with both Positive and Negative. This suggest that the model may rely on features that are not distinctive enough to clearly distinguish sentiment categories with mild/ambiguous emotional tone.

FACTORS CONTRIBUTING TO THE MODEL'S STRENGTHS

1. Use of TF-IDF Vectorization - The use of TF-IDF transforms the tweets into meaningful numerical vectors in a way that boosts the impact of class-specific words, such as 'terrible' or 'excellent, making Negative and Positive tweets easier to classify.
2. Sufficient and Balanced Sample Size - Our dataset has relatively balanced values across classes, helping the model learn evenly from each sentiment type. Logistic Regression works well when it receives enough examples for each class (Statistics Solutions, 2025), as indicated by the high performance on the Negative and Positive classes.

3. Use of Logistic Regression - The use of Logistic Regress is well suited for our TF-IDF data. This is because is it very efficient with word vectors, contributing to the model's overall reliability.

FACTORS CONTRIBUTING TO THE MODEL'S WEAKNESSES

1. Ambiguous Neutral Tweets - Neutral tweets often contain ambiguous or emotionally flat language and tone. This makes them difficult to separate from mildly positive or negative statements, causing frequent misclassification.
2. TF-IDF Disregards Important Linguistic Features of Tweets - A downside of usign TF-IDF is that only word frequency is captured, while intent, tone, context, and word order are ignored. This limitation makes it hard for the model to understand phrases with sarcasm, negation, and even words that may appear in both Neutral and Positive contexts (like "okay" or "good").
3. Limited Examples of Weakly Expressed Sentiment - The training data may lack enough examples of weakly expressed sentiments, such as mildly positive or neutrally sarcastic. This means the model cannot learn how to handle these tweets effective and may make it bias toward Positive and Negative when encountering such statements (contributing to Neutral's lower recall).

DISCUSS POTENTIAL IMPROVEMENTS

1. Represent Features More Accurately (Not relying on word count alone) - N-grams are efficient ways to capture language patterns, structure, an even contextual relationships between adjacent words fro better feature representation (Marketing Mind, 2025). Moreover, Parts-of-Speach tagging and negation handling can also help the model comprehend the semantics behind every tweet and understand the human language more accurately (Lal, 2022; GeeksForGeeks, 2024c).
2. Trying More Advanced Models - Another way to improve classification is using more advanced models like Support Vector Machines (SVM) or Ensemble models.
3. Improving Training Data - Lastly, we can also add more data (especially for the Neutral class) or clean the data better. We can also use techniques like paraphrasing and synonym replacement to help the model learn better.

Source: Kaggle - Twitter Sentiment Analysis by Passionate-NLP

REFERENCES

PART 1

- Pandas. (2024a). pandas.DataFrame.info — pandas 2.2.3 documentation. Pandas.
- Pandas. (2024b). pandas.DataFrame.reset_index — pandas 2.2.3 documentation. Pandas.
- Pandas. (2024c). pandas.DataFrame.shape — pandas 2.2.3 documentation. Pandas.
- W3Schools. (2025a). Pandas DataFrames. W3Schools.
- W3Schools. (2025b). Python String lower() Method. W3Schools.

PART 2

- GeeksforGeeks. (2024, July 12). How to store a TfidfVectorizer for future use in scikit-learn?. GeeksforGeeks.

- Geeksforgeeks. (2024a, January 3). Removing stop words with NLTK in Python. GeeksforGeeks.
- Geeksforgeeks. (2024b, May 9). Understanding Logistic Regression. GeeksforGeeks.
- Karabiber, F. (2024). TF-IDF — Term Frequency-Inverse Document Frequency. LearnDataSci.
- Khanna, C. (2020, August 25). What and why behind fit_transform() vs transform() in scikit-learn. Towards Data Science
- NLTK. (2009). Natural Language Toolkit — NLTK 3.4.4 documentation. NLTK.
- Vanga, S. (2024, February 4). Text Preprocessing in Machine Learning: Finding the Hidden Value of Clean Data. Medium.
- Scikit Learn. (2018). sklearn.model_selection.train_test_split — scikit-learn 0.20.3 documentation. Scikit-Learn.
- Scikit Learn. (2025). LogisticRegression. Scikit-Learn.
- Yang, A. (2020, January 7). Tutorial: Python Regex (Regular Expressions) for Data Scientists. Dataquest.

PART 3

- Abba, I. (2023, January 12). Matplotlib Figure Size – How to Change Plot Size in Python with plt.figsize(). freeCodeCamp.
- GeeksForGeeks. (2024c, January 3). POS(Parts-Of-Speech) Tagging in NLP. GeeksforGeeks.
- GeeksForGeeks. (2024d, July 4). Introduction to Seaborn – Python. GeeksforGeeks.
- GeeksForGeeks. (2025, May 16). Understanding the Confusion Matrix in Machine Learning. GeeksforGeeks.
- Lal, U. (2022, May 27). Increasing Accuracy of Sentiment Classification Using Negation Handling. Towards Data Science.
- Lexanalytics. (2024). Sentiment Accuracy: Explaining the Baseline and How to Test It. Lexanalytics.
- Marketing Mind. (2025). N-grams, Bigrams and Trigrams - Natural Language Processing. Marketing Analytics Solutions.
- Seaborn. (2024). seaborn.heatmap — seaborn 0.13.2 documentation. Seaborn.
- Statistics Solutions. (2025). Assumptions of Logistic Regression. Statistics Solutions.
- W3Schools. (2025c). Matplotlib Pyplot. W3Schools.